COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

Module 28: Game AI, decision systems and machine learning

Four expanded lessons • Project and review

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Module 28: Game AI, decision systems and machine learning 4

Lesson 28.1: Navigation graphs, steering and dynamic obstacles . . . 5

Lesson 28.2: Behavior trees, utility systems and explainable decisions . . . 8

Lesson 28.3: Adversarial search, planning and simulation budgets . . . 11

Lesson 28.4: Learning systems, neural inference and responsible evaluation . . . 14

Module 28 project . . . 16

Module 28 review . . . 17

Reference Directory 18

S01: Collision documentation . . . 18

S02: Simulation documentation . . . 18

S03: glTF tutorial: Skins . . . 18

S04: glTF 2.0 specification . . . 18

S05: Writing a callback function . . . 18

S06: SDL_AudioStreamCallback . . . 18

S07: Xbox Accessibility Guidelines . . . 18

S08: CS188: Informed search . . . 19

S09: Introduction . . . 19

S10: CS188: Minimax . . . 19

S11: Automatic differentiation with torch.autograd . . . 19

S12: Networked movement in the Character Movement Component . . . 19

S13: Snapshot Interpolation . . . 19

S14: GGPO SDK repository 19

S15: Transaction documentation 20

S16: Atomic Commit in SQLite 20

S17: os.replace documentation 20

S18: zipfile documentation 20

S19: Fix Your Timestep! 20

S23: XAG 103: Additional channels for audio and visual cues 20

S24: Deterministic Lockstep 20

S25: XAG 107: Input 20

S26: XAG 101: Text display 21

Original program reading IDs 21

Module 28: Game AI, decision systems and machine learning

Prerequisites: Modules 09, 14, 24 and 26.

Original reading IDs: R20, R46, R47. See the reference directory.

Module goal: Build an inspectable agent that navigates, chooses actions and recovers from blocked plans. Compare a rules-based baseline with one small learned or search-based component; evaluate behavior and runtime cost, not only win rate.

Lesson 28.1: Navigation graphs, steering and dynamic obstacles

Learning goal: Separate finding a route from executing movement.

Planning and movement solve different problems

Pathfinding answers which route connects a start and a goal in a model. Steering answers how an actor moves along that route under speed, acceleration and collision constraints. A perfect route through point-sized graph nodes can be impossible for a wide character. A local steering controller can avoid a nearby obstacle yet fail to discover a route around a large wall. Keep both layers inspectable.

Define a navigation state: grid cell, graph vertex or polygon. Define legal transitions and their costs. For the core grid, movement is four-connected with unit cost; Manhattan distance is an admissible and consistent heuristic. Adding diagonal moves, terrain multipliers or teleport edges changes the assumptions. A heuristic that overestimates can sacrifice shortest-path guarantees. Berkeley's informed-search material is the assigned algorithm reference [S08].

A* with an explicit budget

A* orders the frontier by f=g+h, where g is the best known cost from the start and h estimates remaining cost. Maintain a best-cost map and predecessor map. A heap can contain stale entries after a better route is discovered; skip an entry whose g no longer matches the best known cost. Stable tie-breaking makes traces easier to reproduce. If using a merely admissible but inconsistent heuristic, a graph-search implementation may need node reopening; do not assume a closed set is always safe.

A node-expansion budget is reproducible and useful for teaching, but it is not a wall-clock guarantee if one expansion has unbounded neighbor work. Bound degree and data sizes too. Return a distinct status for found, unreachable and budget_exhausted. Confusing an interrupted search with a proof of no path produces bad gameplay decisions. An incremental planner may retain frontier state across ticks; a simpler lab can stop and use a safe idle action.

Geometry belongs in the navigation model

A 1.0 m wide agent cannot pass a 0.8 m doorway under a no-overlap model. Inflate obstacles by the agent radius or erode walkable space by its footprint. Different agent sizes may need different navigation data or clearance tests. For a grid, make clear whether blocked cells represent centers or occupied areas. A path's graph validity does not prove that every smoothed segment has geometric clearance.

Dynamic obstacles create a versioning problem. Tag the path with the navigation revision and revalidate upcoming segments. Replanning every frame for every unit can overwhelm an RTS. Use blocked-path detection, staggered work, a global expansion budget and a queue with explicit priority. Avoid starvation by recording wait time, not only distance to the camera.

Worked path and budget

On an empty four-connected grid, start (0,0) and goal (3,2) have Manhattan distance 5. Every shortest path has five moves. If a wall blocks (1,0), (1,1) and (1,2), a route may need to go around it if the grid extends far enough; if the grid is only three rows high and the wall spans its full height, no route exists. A budget of two expansions can stop before either result is known. Report budget_exhausted rather than unreachable.

For steering, let desired velocity point toward the next waypoint at a capped speed. Limit velocity change by max_acceleration*dt. Advance the body through collision resolution rather than assigning the target position directly. An arrival radius avoids oscillation around an exact point, but a radius larger than a narrow doorway can skip an essential waypoint. Match steering tolerances to geometry and speed.

A useful debug view

Display blocked cells, agent footprint, planned path, current waypoint, desired velocity, actual velocity and search status. Log when a path becomes invalid and why. A stationary unit can be stuck because the planner returned no path, the steering acceleration is zero, collision rejects motion or a behavior action was canceled. One final position does not distinguish those failures.

The exercise kernel implements bounded A* over a supplied grid. Its reference is intentionally small. The full lab adds footprint clearance, steering, a dynamic obstacle and an unreachable-goal test. Measure expansions and elapsed time separately. Compare the A* path cost to a breadth-first or Dijkstra oracle on small maps, not to the same A* implementation with a different display setting.

Engine integration

Navigation workers consume an immutable world snapshot or versioned query interface. They return a path plus the revision used to compute it. Before installing the result, check that the requesting entity still exists and that its goal generation matches. Otherwise an old asynchronous result can steer a new entity that reused the same index. This is the same lifetime discipline introduced for physics and animation resources.

Original worked checkpoint

A doorway 0.8 units wide is not traversable by a circular agent with diameter 1.0 under a no-overlap model, even if the navigation graph contains an edge through its center.

Full practical assignment

Combine A* with a steering controller and a blocked-path detector. Pass: agent size is respected, unreachable goals terminate or report failure, and repeated replanning has a bounded budget.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Budgeted four-connected A-star**. Run `starter/python/demos/lesson_28_1.py`; implement `starter/python/exercises/lesson_28_1.py`.

Assigned study and native transfer

R46 : Search; R20 : game AI and navigation concepts.

Read S08 on A*, admissibility and consistency. Revisit Modules 09 and 14 for graphs, heaps and complexity. Compare one tiny map against an independent shortest-path oracle.

Engine transfer: Add a globally budgeted navigation queue and stale-result rejection to the engine. Demonstrate a wide and a narrow actor receiving different valid routes.

Check your understanding

1. How do pathfinding and steering differ?
2. Can the example 1.0-wide agent pass through the 0.8-wide doorway?
3. Why bound replanning work?
4. For a four-connected unit-cost grid, what is the Manhattan heuristic from (0,0) to (3,2)? Does a two-expansion budget prove a route is impossible?
5. A smoothed path segment cuts across an obstacle corner although every original grid node is free. What validation is missing?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 28.2: Behavior trees, utility systems and explainable decisions

Learning goal: Represent behavior so designers can understand its failures.

Make the reason for a decision observable

An AI that looks intelligent in one recording may be impossible to maintain. Designers need to know which condition selected an action, what data it used and why the previous action stopped. Represent perception, decision and execution separately. A sensor updates facts; a selector chooses an action; the action owns multi-tick work and cleanup.

A finite-state machine is often sufficient for a small game. A behavior tree composes conditions and actions. A sequence generally advances while children succeed and returns on failure or running; a selector generally tries alternatives until one succeeds or runs. Reactive and memory variants differ in whether they revisit earlier children. Those semantics must be explicit. BehaviorTree.CPP's own documentation emphasizes nonblocking actions and state-transition logging [S09]. The core lesson does not require that library.

Running is a real state

An action that requests a path cannot report success merely because it started. It remains running until the request completes and the destination is reached, or fails when the goal becomes invalid. Cancellation must release its reservations, unsubscribe callbacks and invalidate outstanding generations. A canceled action that later receives a path result must not resurrect itself.

Define a lifecycle: enter, tick, success/failure, and cancel/exit. An action should release a resource exactly once even if cancellation races conceptually with completion. Use ownership tokens or generation checks rather than loosely paired booleans. Keep the decision trace bounded; a ring of the last few hundred records is more useful than an unbounded log in a shipping build.

Utility scores and hysteresis

A utility system computes comparable scores for candidate actions. Normalize feature ranges and document their meaning. Mixing health on a 0..100 scale with distance on a 0..1 scale gives arbitrary weight to units. An action may be invalid regardless of its score; filter invalid candidates before selection.

Pure argmax can oscillate when scores are nearly tied. Introduce a switching margin, a minimum commitment time or both. For a margin m, retain the current valid action unless best_score > current_score + m. Emergency actions can bypass ordinary commitment. Hysteresis trades immediate responsiveness for stability; a margin is not a universal solution and must be tuned against actual transitions.

Worked selector trace

Suppose attack has utility 0.61, retreat 0.60 and patrol 0.20. The actor is retreating and the switch margin is 0.05. Attack does not exceed retreat by more than 0.05, so retreat remains active. On the next tick, health falls and retreat rises to 0.90; the selector still retains retreat for a different reason. Later attack rises to 0.70 while retreat falls to 0.50, so switching is permitted. Record scores, validity, current action, chosen action and the reason.

If the current action becomes invalid, do not retain it merely because the margin is large. With no valid actions, choose a safe idle/failure state and emit a diagnostic. Treat NaN utility as invalid input rather than letting language-specific comparison behavior select a mysterious winner. Stable action IDs break exact ties deterministically.

Blackboard discipline

A blackboard stores shared facts, but an unrestricted global dictionary can hide dependencies. Give entries types, owners and freshness. A target position from five seconds ago is different from a current observation. A last-seen target can be a deliberate memory feature, but label its timestamp and uncertainty. Do not let AI read hidden authoritative enemy state that its game design says it cannot know.

For concurrent jobs, decisions should consume a coherent snapshot. A mix of this tick's health and last tick's target ownership can create impossible combinations. Limit mutation to a known phase or version the input facts. Log the fact revision with the decision so failures can be replayed.

Deliberate practice

The exercise implements a deterministic utility selector with hysteresis. The full lab adds at least one multi-tick action and three induced failures: unreachable target, canceled resource owner and missing/stale perception. Explain each failure through a trace, not a print statement saying only 'AI failed.' Test that cancellation frees its resource and that an old callback cannot re-enter the action.

For the capstone, prefer one inspectable behavior that reliably interacts with the finished gameplay loop over many opaque behaviors. A rules-based patrol/repair/retreat controller can be excellent engineering evidence. Machine learning in the next lesson is an alternative tool, not a requirement to replace understandable decisions everywhere.

Original worked checkpoint

An agent choosing between attack score 0.51 and retreat score 0.50 can oscillate as small observations change. A switching threshold or minimum commitment interval can stabilize the policy, with explicit tradeoffs.

Full practical assignment

Implement either a small behavior tree or utility selector with a trace viewer. Pass: action cancellation releases resources and three induced failure cases produce understandable explanations.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Deterministic utility selection with hysteresis**. Run `starter/python/demos/lesson_28_2.py`; implement `starter/python/exercises/lesson_28_2.py`.

Assigned study and native transfer

R20 : game AI; R46 : agents, utilities and decision-making foundations.

Read S09 and its linked basic behavior-tree concepts. Compare sequence/selector semantics with your own explicit lifecycle. Revisit Module 24 ownership and event queues.

Engine transfer: Expose a bounded decision trace in the engine inspector, including facts, scores, chosen action, cancellation reason and outstanding request generation.

Check your understanding

1. What statuses should a multi-frame action expose?
2. Why can nearly tied utility scores cause oscillation?
3. Why is a decision trace valuable?
4. The current action scores 0.60, the best alternative scores 0.64 and the switching margin is 0.05. Which action remains, assuming both are valid?
5. A canceled navigation action receives its asynchronous result later and starts moving again. What state or ownership check should prevent this?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 28.3: Adversarial search, planning and simulation budgets

Learning goal: Trade decision quality against bounded computation.

Search is a budgeted policy, not a magical opponent

For a deterministic, turn-taking, zero-sum, perfect-information game, minimax chooses the move with the best outcome against an optimal opponent. At maximizing nodes, take the largest child value; at minimizing nodes, take the smallest. Terminal values are game-defined outcomes. Depth-limited evaluation estimates nonterminal states and therefore changes the strength of the guarantee. Berkeley's minimax chapter is the formal reading [S10].

An RTS is often simultaneous, partially observed and much larger than the tiny turn-taking model. A small search can still help choose a tactical action, but state clearly what the internal model omits. Do not claim that solving a toy minimax tree solves full RTS strategy. The capstone needs one bounded, inspectable decision system, not an unrestricted strategist.

Alpha-beta preserves the search objective

Alpha is a lower bound already available to the maximizer; beta is an upper bound already available to the minimizer. When a branch cannot improve the ancestor's choice, stop exploring it. With correct implementation and the same depth/evaluation model, alpha-beta returns the same minimax value while potentially visiting fewer nodes. Move ordering affects how much pruning occurs, not which fully searched objective is being optimized.

For a root maximizer whose left minimizing child has leaves [3,5], the left value is 3. Now inspect a right minimizing child with first leaf 2. Its final value cannot exceed 2, so the root already prefers the left branch and the remaining right leaves can be pruned. This is a proof about bounds, not a heuristic guess that the remaining leaves are probably uninteresting.

Exponential growth and iterative deepening

A branching factor of 10 at depth 6 gives about one million leaf paths, before counting internal nodes. A time budget alone is hard to reproduce across machines. Record both elapsed time and expanded nodes. In the teaching harness, a node budget ensures a bounded search workload given bounded move generation. In a runtime system, also bound per-node operations and check a deadline at useful intervals.

Iterative deepening completes depth 1, then 2, and so on. Retain the last fully completed root result. If depth 4 is interrupted, do not label its partly evaluated root as equivalent to a completed depth-4 search. Before searching, choose a legal fallback action so the actor always has a valid response. A zero budget should return that fallback, not an uninitialized move index.

Worked budget decision

A small game offers moves 'take one' and 'take two' from a pile. Taking the last item wins. With a pile of 4, taking one leaves 3, which is losing for an optimal opponent in this rule set. A complete search can prove the move. With a budget too small to finish any root depth, the system returns its declared legal fallback. The fallback may be strategically poor, but it is not a correctness failure unless the API promised optimality under that budget.

Separate the result fields: chosen_move, completed_depth, expanded_nodes, budget_exhausted and value_if_complete. That distinction makes performance-quality experiments meaningful. A result that silently replaces 'unknown' with zero can bias action selection toward branches that happened to time out.

Planning alternatives and evaluation

Monte Carlo tree search estimates action quality through simulations; it also needs rollout, exploration and budget policies. A planner with action preconditions and effects may fit a resource-management problem better than minimax. Choose the model that matches the decision, and compare to a simple baseline. A deeper but inaccurate internal simulation can be worse than a shallow model with better assumptions.

Test multiple seeds, starting states and opponents. A single self-play win rate can reward shared bugs or a narrow strategy. Count illegal actions, timeouts, nodes, latency and outcome quality separately. Keep a deterministic tie-break so regressions can be traced. When an evaluation function changes, preserve the test corpus rather than choosing new easy positions.

Deliberate practice

The kernel operates on a small explicit minimax tree with alpha-beta and a strict node cap. The full lab builds a tiny legal-move game, adds iterative deepening and a fallback, and compares against an exhaustive solver for small states. Induce a zero budget, a terminal root, an exact tie and an interrupted deeper iteration. For an engine transfer, demonstrate that a decision deadline cannot stall the simulation indefinitely and that the trace reports incomplete search honestly.

Original worked checkpoint

A branching factor of 10 searched to depth 6 can produce roughly one million leaf paths before pruning. A 2 ms decision budget cannot be assumed sufficient without measurement and a bounded policy.

Full practical assignment

Implement minimax with alpha-beta or a small MCTS agent for a tiny game. Pass: legal moves are always returned within a defined budget or fallback, and evaluation includes multiple seeds and opponents.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Node-bounded alpha-beta value search**. Run

`starter/python/demos/lesson_28_3.py`; implement `starter/python/exercises/lesson_28_3.py`.

Assigned study and native transfer

R46 : Games, Minimax, Expectimax and Monte Carlo Tree Search.

Read S10 on minimax, alpha-beta and evaluation functions. Derive one pruning decision by hand. Use the original Module 14 complexity tools to bound your move generator.

Engine transfer: Integrate a decision job with a legal fallback and explicit completion status. Record how increasing the budget changes both response latency and decision quality.

Check your understanding

1. Does alpha-beta change the minimax objective when applied correctly?
2. How many depth-6 paths arise from branching factor 10 without pruning?
3. Why use a fallback action?
4. A root maximizer has already found value 3. A new minimizing child first reveals leaf value 2. Why may its remaining leaves be pruned?
5. Depth 3 completed but depth 4 exhausts its node budget halfway through the root. Which result should iterative deepening return, and what should be reported?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 28.4: Learning systems, neural inference and responsible evaluation

Learning goal: Understand learning fundamentals without replacing every subsystem with a model.

Start with a question a small model can answer

A useful learning task might predict whether a unit should retreat from features such as normalized health, nearby threat and available cover. Define the target, label-generation procedure and evaluation conditions before training. A model that predicts a rule-generated label may simply approximate that rule; success does not demonstrate discovery of superior strategy. Keep the rule itself as a baseline.

Training changes parameters using examples. Inference applies frozen parameters to new inputs. A game must budget inference time, model memory, feature construction and fallback behavior. Training speed is not the runtime metric. The core exercise uses a small linear logistic classifier implemented with the standard library so the arithmetic is visible and no GPU is required.

Derive one gradient step

For features x and parameters w,b, compute z=w dot x+b and p=1/(1+exp(-z)). For binary label y, cross-entropy loss is -y*log(p)-(1-y)*log(1-p). Combining the derivative of the sigmoid with the derivative of this loss gives dL/dz=p-y. Therefore dL/dw=(p-y)*x and dL/db=p-y. This simplification is an application of the chain rule. Automatic differentiation generalizes that bookkeeping through larger computation graphs [S11].

Use a numerically stable sigmoid branch for large positive and negative z, and a stable loss expression rather than taking log(0). Reject non-finite features. Standardize or bound features using statistics fit only on training data. A confidence-like score from a classifier is not automatically a calibrated probability under a new game distribution.

Worked update

Let x=(1,2), y=1, initial w=(0,0), b=0 and learning rate 0.1. Then z=0 and p=0.5. The weight gradient is (-0.5,-1) and the bias gradient is -0.5. One gradient-descent step gives w=(0.05,0.10) and b=0.05. The new logit is 0.30 and probability about 0.5744. This is a one-example update, not evidence that the model generalizes.

Finite-difference checks on a tiny parameter vector can verify gradients. Compare the analytic derivative to [L(w+epsilon)-L(w-epsilon)]/(2*epsilon), using a sensible epsilon and a toleranced comparison. A small training loss does not validate a wrong gradient if the task is trivial or the implementation leaks labels.

Split by the source of correlation

Frames from one episode are correlated. Randomly splitting individual frames can place nearly identical situations from the same trajectory in both training and test sets. Split complete episodes, and sometimes maps, players or seeds, according to the intended generalization question. Keep a validation set for model selection and an untouched test set for the final estimate. Never tune a threshold repeatedly on the reported test set and still call it unseen.

Record episode IDs, data-generation seeds, feature schema, model parameters, optimizer settings and the split rule. Synthetic labels should be documented as synthetic. Avoid collecting personal player data when a synthetic task satisfies the educational goal. Report class balance and a confusion matrix so a model that always selects the majority action cannot hide behind accuracy alone.

Deployment quality includes failure behavior

Test an out-of-range health value, missing feature, unsupported model version, corrupted weights and an inference budget overrun. A safe fallback should still return a legal game action. For a tactical model, a rules-based conservative action may be appropriate. Model confidence alone should not override authoritative movement or reward validation.

Measure feature construction and inference separately. Run enough repeated calls to reduce timer granularity effects, but report batch size and distinguish batched throughput from one-agent latency. A toy Python measurement is not a claim about native engine performance. The full lab must compare a frozen model and the rule baseline on independent episodes, include an error analysis and measure the selected deployment path.

Responsible interpretation

A learned policy is not necessarily more explainable, fairer or more efficient than a rule system. Evaluate the purpose it actually serves. In the capstone, a simple authored AI remains acceptable; the learning lab is still required as a comparison exercise. The learner should be able to explain when training is worth its data and maintenance costs and when a transparent rule wins. A negative result is valid engineering evidence, provided the experiment and baseline are sound.

Original worked checkpoint

A model trained and tested on adjacent frames from the same recorded match may appear accurate by exploiting near-duplicates. Holding out complete matches provides a stronger test of generalization.

Full practical assignment

Train a small classifier or value estimator on synthetic game data and compare it with a rules baseline. Pass: independent splits, error analysis, inference timing, reproducible seeds and a fallback behavior are reported.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **One stable logistic gradient step**. Run `starter/python/demos/lesson_28_4.py`; implement `starter/python/exercises/lesson_28_4.py`.

Assigned study and native transfer

R46 : Machine Learning and Reinforcement Learning; R47 : linear models, multilayer perceptrons, backpropagation and generalization.

Read S11 on computation graphs and automatic differentiation. Use the original probability/statistics material to review evaluation and uncertainty. No PyTorch dependency is required for the core kernel.

Engine transfer: Export a versioned model and feature schema, then integrate a bounded inference adapter with a rules fallback. Keep model output advisory; authoritative simulation still validates legal actions.

Check your understanding

1. Why separate complete episodes across data splits?
2. What mathematical rule underlies backpropagation?
3. Why retain a rules-based baseline?
4. For x=(1,2), y=1, w=(0,0), b=0 and learning rate 0.1, compute one logistic cross-entropy gradient-descent update.
5. A classifier scores 98% when frames are randomly split, but 72% when entire episodes are held out. Which estimate better supports new-episode use, and why?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 28 project

Build an Inspectable Courier Agent. The agent travels to a target, responds to a blocked route and chooses among a small set of understandable actions. Give every decision a readable trace, including invalid alternatives and action cancellation. Use a strict search/replan budget and a safe behavior when it is exhausted.

Compare a rules baseline with one search-based or learned component. The lesson on learning still requires its synthetic training/evaluation exercise even when the integrated agent uses search. Select evaluation scenarios before looking at results: a clear route, a narrow corridor, a newly blocked passage, a missing target and at least one adversarial case.

Report completion/failure categories, time or work per decision, cancellation/resource behavior and reproducible seeds. For learned behavior, use independent episode splits and a frozen artifact. For search, provide legal-move/route checks and a completed-iteration fallback. A baseline outperforming the sophisticated component is useful engineering

evidence.

Deliver a behavior specification, scenario manifest, raw traces, oracle comparisons, cost distributions and a short design decision explaining the chosen approach. A reviewer should cause a failure and explain the recovery from the trace without reading every source line.

Module 28 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A* found a route, but the agent never reaches the target. Name four different layers to inspect.
2. How do hysteresis and action cancellation solve different AI problems?
3. A search expires before completing the first root move. What can it safely return?
4. A classifier scores 99% after randomly splitting frames from the same matches. Why is that evidence weak?
5. How should a learned component behave when the model fails to load or inference is too slow?

Reference Directory

S-prefixed entries are the focused primary-source companions consulted for this expansion, September 28, 2026. Numbers are stable identifiers; gaps are intentional. R-prefixed entries preserve the original program reading directory. External books, papers, documentation and library source are not bundled; the explanations, exercises and demonstration code in this package are original teaching material. Pin installed versions in your own evidence rather than treating a mutable documentation page as an installed build.

S01: Collision documentation

Box2D project. Shape queries, broad phase, dynamic tree and time-of-impact limitations.

Open primary source

S02: Simulation documentation

Box2D project. Stepping, contacts, solver settings and the documented determinism contract.

Open primary source

S03: glTF tutorial: Skins

Khronos Group. Joint transforms, inverse bind matrices and mesh-node correction.

Open primary source

S04: glTF 2.0 specification

Khronos Group. Normative asset, animation and skinning data contracts.

Open primary source

S05: Writing a callback function

PortAudio project. Callback constraints and real-time-safe execution discipline.

Open primary source

S06: SDL_AudioStreamCallback

SDL project. SDL3 audio stream callback contract; consult the API for the chosen integration.

Open primary source

S07: Xbox Accessibility Guidelines

Microsoft. Game accessibility design/testing guidance, not a legal-compliance certification.

Open primary source

S08: CS188: Informed search

University of California, Berkeley. A*, heuristics and search guarantees under stated assumptions.

Open primary source

S09: Introduction

BehaviorTree.CPP project. Behavior-tree concepts, statuses and implementation context.

Open primary source

S10: CS188: Minimax

University of California, Berkeley. Adversarial search and alpha-beta reasoning.

Open primary source

S11: Automatic differentiation with torch.autograd

PyTorch project. Gradient/parameter concepts; framework use is optional for the core synthetic exercise.

Open primary source

S12: Networked movement in the Character Movement Component

Epic Games. A maintained engine comparison for authority, prediction and correction.

Open primary source

S13: Snapshot Interpolation

Glenn Fiedler. Author’s technical explanation of interpolation, jitter and visual delay.

Open primary source

S14: GGPO SDK repository

GGPO authors. Primary implementation/documentation for rollback networking concepts.

Open primary source

S15: Transaction documentation

SQLite project. Transaction boundaries and behavior.

Open primary source

S16: Atomic Commit in SQLite

SQLite project. Atomicity assumptions and storage failure considerations.

Open primary source

S17: os.replace documentation

Python Software Foundation. Replacement semantics and platform/filesystem conditions.

Open primary source

S18: zipfile documentation

Python Software Foundation. Archive processing and extraction boundaries.

Open primary source

S19: Fix Your Timestep!

Glenn Fiedler. Simulation/render time separation and work-budget hazards.

Open primary source

S23: XAG 103: Additional channels for audio and visual cues

Microsoft. Critical information through more than one channel.

Open primary source

S24: Deterministic Lockstep

Glenn Fiedler. Input synchronization and deterministic simulation requirements.

Open primary source

S25: XAG 107: Input

Microsoft. Input/remapping accessibility considerations.

Open primary source

S26: XAG 101: Text display

Microsoft. Readable interface text and testing considerations.

Open primary source

Original program reading IDs

These inherited assignments remain part of the full study route. S entries above focus the expanded lesson tasks; they do not replace all external study.

R17: RFC 9293: Transmission Control Protocol — IETF / RFC Editor. Transport semantics; reference reading after networking foundations. Assigned source

R19: 15-462/662 Computer Graphics — Carnegie Mellon University. Rendering, geometry and animation course structure and exercises. Assigned source

R20: 15-466/666 Computer Game Programming — Carnegie Mellon University. Runtime systems and game-production programming. Assigned source

R22: SDL3 documentation — SDL project. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples. Assigned source

R23: Physically Based Rendering, fourth edition — Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments. Assigned source

R26: Programming with C++ in Unreal Engine — Epic Games. Engine comparison after C++ foundations, not a beginner substitute. Assigned source

R28: Box2D documentation — Box2D project. Collision, constraints and simulation reference. Assigned source

R29: Xbox Accessibility Guidelines — Microsoft. Accessibility design and testing; not a legal-compliance certificate. Assigned source

R45: AddressSanitizer documentation — LLVM Project. Memory-error instrumentation, limitations and supported toolchains. Assigned source

R46: CS188 Introduction to Artificial Intelligence textbook — University of California, Berkeley. Search, games, decision processes, learning and probabilistic reasoning. Assigned source

R47: Dive into Deep Learning — Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola. Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions. Assigned source